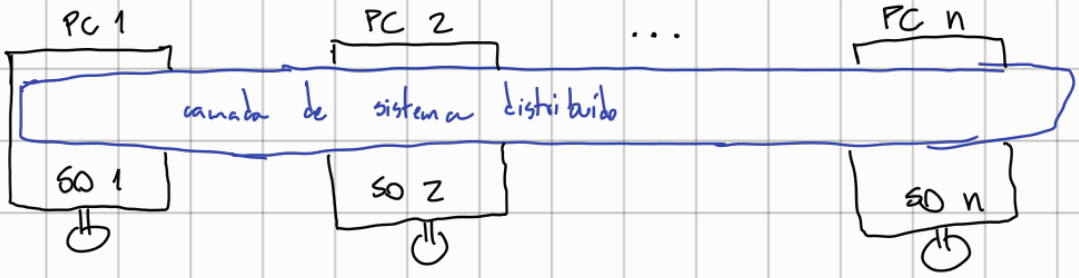
Aula 2 - Sistemas Operacionais e SD

conceitos de SO

classificação de SO (tipos)

- monotarefa
- multitarefa
  - ↳ batch
  - ↳ tempo compartilhado
  - ↳ tempo real
- múltiplas CPU
  - ↳ paralelismo
  - ↳ computação paralela para SDs

sistema distribuído



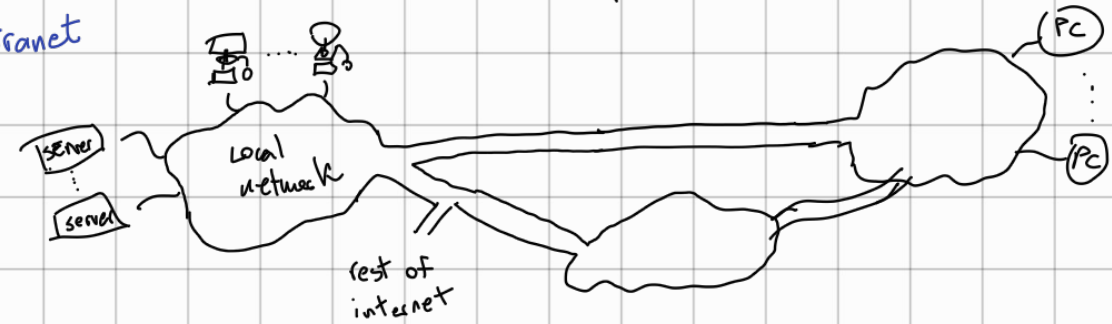
problema dos relógios

- sincronização de relógios
- facilita a construção de sistemas distribuídos
- solução para exclusão mútua

desafios

- segurança
- integridade
- heterogeneidade
- disponibilidade
- identidade do usuário
- escalabilidade
  - ↳ dividir para conquistar

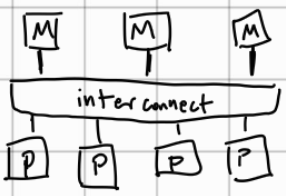
exemplo de SD - intranet



AULA SD - 19/08/2026

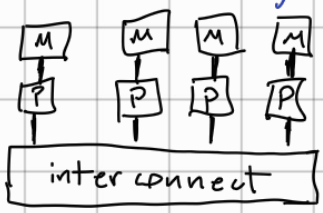
início → computação paralela
↓
computação distribuída

shared memory



↓
Facilidade sem escalabilidade

private memory

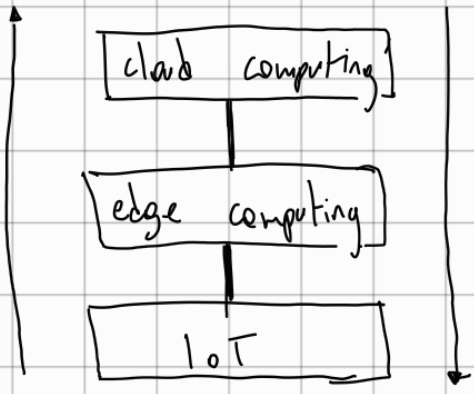


↓
escalabilidade porém exige comunicação

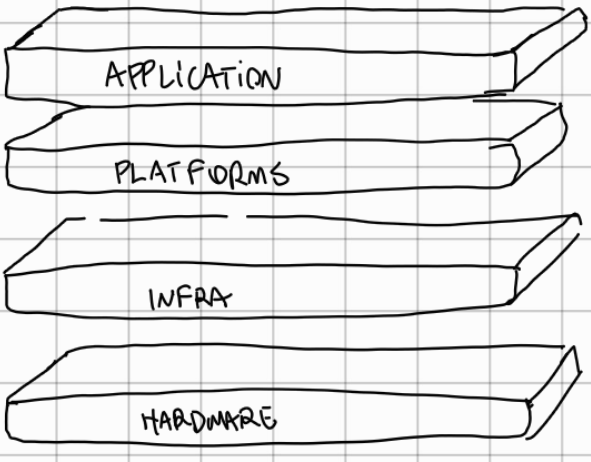
cluster
↳ computadores similares, mesmo SO e com conexão por rede veloz

grid
↳ sistema descentralizado, cada um em um domínio administrativo diferente, podendo ter diferentes hardware e software

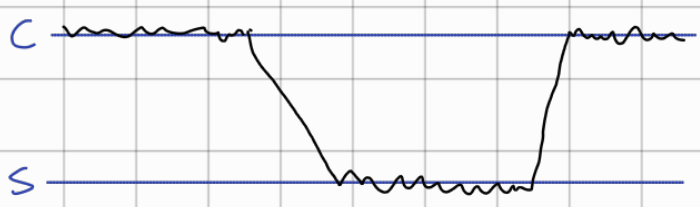
prós e contras



arquiteturas híbridas



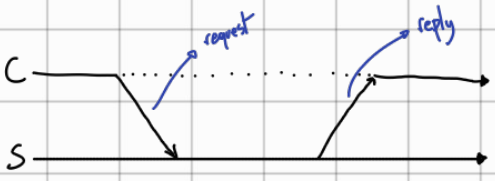
cliente - servidor
C S



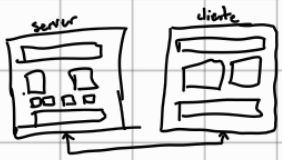
AULA 26/05 - SISTEMAS DISTRIBUÍDOS

arquitetura em camadas

modelo cliente - servidor

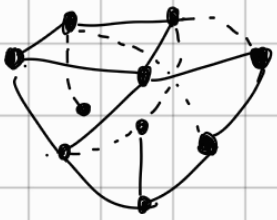


ex.: NFS



peer - to - peer

"pares em pares"



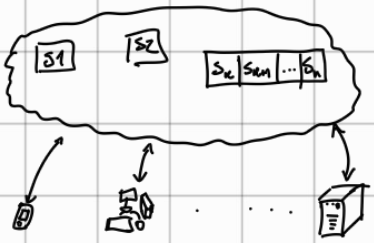
→ todos processos desempenham sendhantemente

→ resistência a falhas

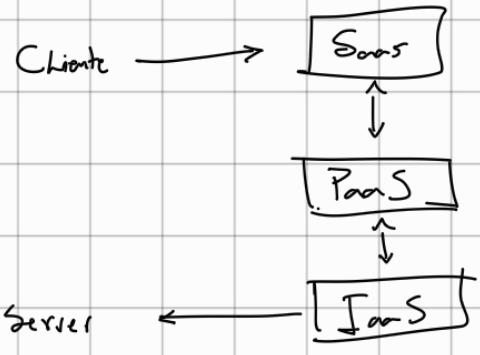
arquiteturas híbridas

computação em nuvem





SaaS e PaaS (platform as a service) e IaaS (infra as a service)



serverless



- serverless
- server contínuo
- não utilizado

tarefa computação sem servidor

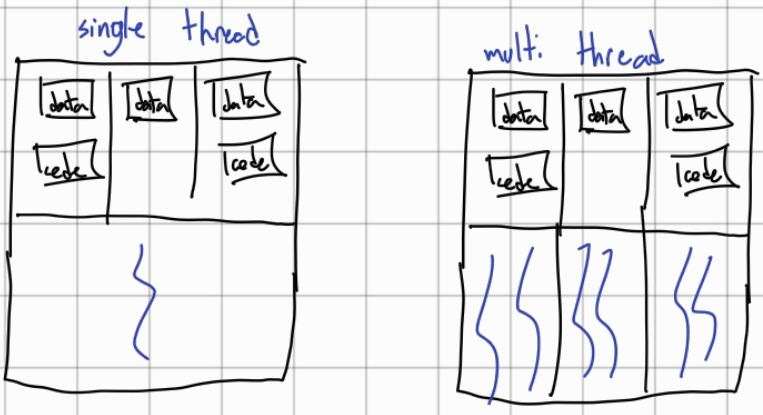
AULA 02/09 - Sistemas Distribuídos PPGCC

processo

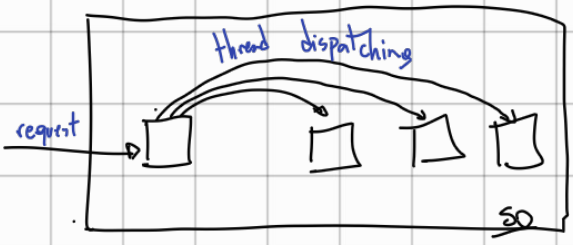
- é um "programa em execução"
- troca de processos ocorre por: salva registradores → mantém o que precisar em disco → troca → executa → interrompe

threads

- são linhas de execução
- 1 processo : N threads



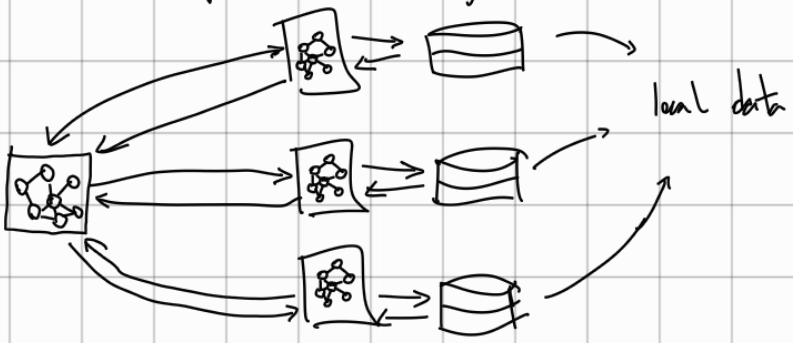
servidor multi-threaded



migração de código

- balanceamento de carga para desempenho
- pode usar diferentes métricas de decisão (fila CPU, utilização CPU...)
- precisa considerar custos: swap e comunicação

federated learning



AULA 09/09 - SISTEMAS DISTRIBUÍDOS

partindo do conceito de processos cliente servidor

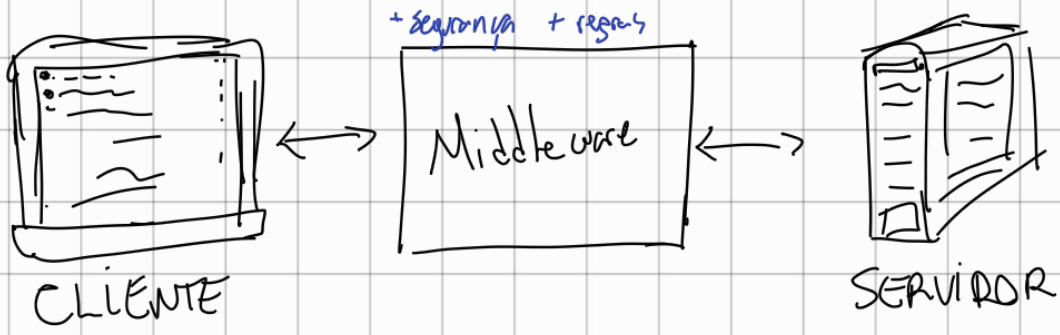
cliente: sw para transparência

- cliente não deve saber o que ocorre por trás "isso não é problema meu"
- software do cliente mascara a ocorrência de falha (reconnect, cache ....)

servidores: organização geral

- iterativo: um pedido por vez, é mais simples, porém um cliente demorado atrasa todos
- concorrente: (recebe cliente → passa para ajudante → livre pro próximo), clientes não ficam esperando
- servidores com estado: guarda o histórico ou dados da sessão do cliente (cache), server cai ↳ dados apagados
- servidores stateless: trata cada requisição como se fosse a primeira, implica em envios repetidos de dados

middleware



tipo de comunicação

- síncrono
  - ↳ assemelha-se ao iterativo, espera-se a confirmação do recebimento da mensagem (TCP)
- assíncrono
  - ↳ mais rápido, sem confirmação de recebimento (UDP)

Remote Procedure Call

